

File: run.py

```
from richdisplay.core import Display, LogControl

if __name__ == '__main__':
    display = Display("richdisplay")

    display.debug("Debug message")
    display.info("Info message")

    display2 = Display("richdisplay2")
    display2.debug("Debug message from display2")
    display2.info("Info message from display2")
```

File: richdisplay/__init__.py

```
from .core import Display, LogControl
```

File: richdisplay/state.py

```
'''
State and Args context management with centralized authentication.
Provides stack-based contexts that rely on a single AuthManager to generate, verify, and revoke tokens.
'''
from dataclasses import dataclass
from threading import local
from typing import Optional, List
import secrets
import uuid

class AuthManager:
    """
    Centralized token management for all contexts.
    """
    _valid_tokens: set[str] = set()

    @classmethod
    def generate_token(cls) -> str:
        """
        Create and register a new secure token.
        """
        auth_token = secrets.token_hex(16)
        cls._valid_tokens.add(auth_token)
        return auth_token

    @classmethod
    def verify_token(cls, auth_token: str) -> bool:
        """
        Check if the token is valid.
        """
        return auth_token in cls._valid_tokens

    @classmethod
    def revoke_token(cls, auth_token: str) -> None:
        """
        Invalidate a token so it can no longer be used.
        """
        cls._valid_tokens.discard(auth_token)

@dataclass(frozen=True)
class StateSnapshot:
    """
    Immutable snapshot of the sys context.
    """
    debug: bool
    log_to_db: bool
```

```
log_level: int
token: str
```

```
@dataclass(frozen=True)
class ArgsSnapshot:
```

```
    """
    Immutable snapshot of the args context.
    """
    clear: bool
    debug: bool
    db: bool
    db_path: str
    log_to_db: bool
    get_logs: int
    export: str
    log_level: int
    token: str
```

```
class BaseContext:
```

```
    """
    Base for context stacks with token verification.
    """
    _local = local()
    _stack_name: str
    _snapshot_type: type

    @classmethod
    def _init_stack(cls):
        if not hasattr(cls._local, cls._stack_name):
            setattr(cls._local, cls._stack_name, []) # type: ignore

    @classmethod
    def _current(cls):
        cls._init_stack()
        stack = getattr(cls._local, cls._stack_name)
        return stack[-1] if stack else None

    @classmethod
    def _push(cls, snapshot):
        cls._init_stack()
        getattr(cls._local, cls._stack_name).append(snapshot)

    @classmethod
    def _pop(cls, auth_token: str):
        cls._init_stack()
        stack = getattr(cls._local, cls._stack_name)
        if not stack:
            raise RuntimeError(f"No {cls.__name__} to pop.")
        current = stack[-1]
        if current.token != auth_token or not AuthManager.verify_token(auth_token):
            raise PermissionError(f"Invalid token for {cls.__name__} pop.")
        stack.pop()
```

```
class SysContext(BaseContext):
```

```
    """
    System-level context for debug and logging state.
    """
    _stack_name = uuid.uuid4().hex

    @classmethod
    def get_debug(cls) -> bool:
        state = cls._current()
        return state.debug if state else False

    @classmethod
    def get_log_to_db(cls) -> bool:
        state = cls._current()
        return state.log_to_db if state else False

    @classmethod
    def get_log_level(cls) -> int:
        state = cls._current()
        return state.log_level if state else 10

    @classmethod
```

```

def push(
    cls, *,
    auth_token: str,
    debug: Optional[bool] = None,
    log_to_db: Optional[bool] = None,
    log_level: Optional[int] = None):
    if not AuthManager.verify_token(auth_token):
        raise PermissionError("Invalid token for SysContext push.")
    base = cls._current() or StateSnapshot(False, False, 1, auth_token)
    debug_flag = debug if debug is not None else base.debug
    level = 0 if debug_flag else (
        log_level if log_level is not None else base.log_level)
    if not isinstance(level, int) or not 10 <= level <= 50:
        level = 10
    snapshot = StateSnapshot(
        debug_flag,
        log_to_db if log_to_db is not None else base.log_to_db,
        level,
        auth_token
    )
    cls._push(snapshot)

@classmethod
def pop(cls, auth_token: str):
    cls._pop(auth_token)

class ArgsContext(BaseContext):
    """
    Argument parsing context for CLI flags.
    """
    _stack_name = uuid.uuid4().hex

    @classmethod
    def get(cls) -> ArgsSnapshot:
        current = cls._current()
        if current:
            return current
        return ArgsSnapshot(False, False, False, 'logs.db', False, 0, '', 10, '')

    @classmethod
    def push(cls, *, auth_token: str, clear: Optional[bool] = None,
        debug: Optional[bool] = None, db: Optional[bool] = None,
        db_path: Optional[str] = None, log_to_db: Optional[bool] = None,
        get_logs: Optional[int] = None, export: Optional[str] = None,
        log_level: Optional[int] = None):
        if not AuthManager.verify_token(auth_token):
            raise PermissionError("Invalid token for ArgsContext push.")
        base = cls._current() or ArgsSnapshot(
            False, False, False, 'logs.db', False, 0, '', 10, auth_token)
        snapshot = ArgsSnapshot(
            clear=clear if clear is not None else base.clear,
            debug=debug if debug is not None else base.debug,
            db=db if db is not None else base.db,
            db_path=db_path if db_path is not None else base.db_path,
            log_to_db=log_to_db if log_to_db is not None else base.log_to_db,
            get_logs=get_logs if get_logs is not None else base.get_logs,
            export=export if export is not None else base.export,
            log_level=log_level if log_level is not None else base.log_level,
            token=auth_token
        )
        cls._push(snapshot)

    @classmethod
    def pop(cls, auth_token: str):
        cls._pop(auth_token)

if __name__ == "__main__":
    # Centralized token handling
    token = AuthManager.generate_token()

    # SysContext demonstration
    SysContext.push(auth_token=token, debug=True)
    print(SysContext.get_debug(), SysContext.get_log_level())
    SysContext.pop(auth_token=token)

    # ArgsContext demonstration

```

```

ArgsContext.push(auth_token=token, debug=True, get_logs=5)
args = ArgsContext.get()
print(args.debug, args.get_logs)
ArgsContext.pop(token)

# Revoke when done
AuthManager.revoke_token(token)

```

File: richdisplay/core/__init__.py

```

from .display import Display
from .log_control import LogControl

__all__ = [
    "Display",
    "LogControl",
]

```

File: richdisplay/core/display.py

```

"""
Handles displaying information in a structured format using Rich library.
"""

import logging
import uuid
import sys
from dataclasses import dataclass
from typing import Optional, Union, List
from rich.logging import RichHandler

from richdisplay.state import SysContext
from richdisplay.utils import RichFormatter
from richdisplay.database import LogDB

from .log_control import LogControl

@dataclass
class DisplayState:
    """Represents the state of the display."""

    _valid_entities: List[str] = [
        "debug",
        "log_to_db",
        "log_level",
        "token",
        "consumer_id",
        "log_ctrl",
    ]

    _debug: bool
    _log_to_db: bool
    _log_level: int
    _token: str
    _consumer_id: str
    _log_ctrl: Optional["LogControl"] = None

    def __init__(
        self,
        log_level: int = 20,
        consumer_id: str = "",
        log_ctrl: Optional["LogControl"] = None,
    ):
        self._log_level = log_level
        self._consumer_id = consumer_id
        self._log_ctrl = log_ctrl

        if self._log_ctrl is None:
            self._log_ctrl = LogControl(self._consumer_id)
            self._log_ctrl.update_log_level(self._log_level)

```

```

    else:
        self._log_ctrl.update_log_level(self._log_level)

def _is_parent(self, pid: str) -> bool:
    """Checks if the current instance is the parent of the given PID."""
    if self._log_ctrl:
        try:
            _ = self._log_ctrl.request_token(pid)
            return True
        except (RuntimeError, PermissionError):
            return False
    return False

def set_entity(self, entity: str, value: Union[str, int], pid: str) -> None:
    """Sets an entity in the DisplayState."""
    if not self._is_parent(pid):
        raise PermissionError(
            f"Permission denied for entity '{entity}' with PID '{pid}'."
        )

    if entity not in self._valid_entities:
        raise ValueError(f"Invalid entity '{entity}' to set.")

    entity = "_" + entity.lower()
    setattr(self, entity, value)

def request_entity(self, entity: str, pid: str) -> Optional[Union[str, int]]:
    """Requests an entity from the DisplayState."""
    if not self._is_parent(pid):
        raise PermissionError(
            f"Permission denied for entity '{entity}' with PID '{pid}'."
        )

    if entity not in self._valid_entities:
        raise ValueError(f"Invalid entity '{entity}' to request.")

    entity = "_" + entity.lower()
    return getattr(self, entity)

```

```

class Display:

```

```

    """Handles displaying information in a structured format."""

    _debug: bool = False
    _log_to_db: bool = False
    _handler: RichHandler
    _token: str = ""
    _log_ctrl: Optional["LogControl"] = None
    _consumer_id: str = ""

    _state: Optional[DisplayState] = None

def __init__(self, logger_name: str, log_lvl: Optional[int] = 20):
    """Initializes the Display instance with a specific logger name."""
    self._consumer_id = uuid.uuid4().hex
    self._log_ctrl = LogControl(self._consumer_id)

    self._state = DisplayState(
        consumer_id=self._consumer_id, log_ctrl=self._log_ctrl, log_level=log_lvl)
    self.logger_name = logger_name

    if log_lvl > 0:
        self._log_ctrl.update_log_level(log_lvl)
    else:
        print(
            "Uh-oh... Logging at level 0 are we? Brace yourselves, the logs are coming!")

    self._logger = logging.getLogger(logger_name)
    self.init_display()

def __del__(self):
    """Destructor to clean up resources."""
    if LogDB._connection:
        LogDB.close_db()

def set_debug(self) -> Union[int, None]:
    """Sets the debug mode for the logger."""
    try:

```

```

        _token = self._state.request_entity("token", self._consumer_id)
        SysContext.push(auth_token=_token)
        _debug = SysContext.get_debug()
        _log_to_db = SysContext.get_log_to_db()
        _log_level = SysContext.get_log_level()

        self._state.set_entity("debug", _debug, self._consumer_id)
        self._state.set_entity("log_to_db", _log_to_db, self._consumer_id)
        self._state.set_entity("log_level", _log_level, self._consumer_id)

    return _log_level
except (RuntimeError, PermissionError) as e:
    print(f"Error: Unable to push system context: {e}")
    sys.exit(1)

def _set_token(self) -> None:
    """Sets the token for the logger."""
    try:
        _log_ctrl = self._state.request_entity(
            "log_ctrl", self._consumer_id)
        _token = _log_ctrl.request_token(self._consumer_id)
        self._state.set_entity("token", _token, self._consumer_id)
        if _token == "":
            raise RuntimeError("Empty token received.")
    except (RuntimeError, PermissionError) as e:
        print(f"Error: Unable to push system context: {e}")
        sys.exit(1)

def init_display(self):
    """Initializes the display settings using the new SysContext push API."""
    # push system context using this instance's token

    self._set_token()

    _log_level = self.set_debug()

    if self._log_to_db:
        LogDB.init_db()

    # Initialize the RichHandler with proper settings
    self._handler = RichHandler(
        show_time=True, # Show timestamp
        show_level=True, # Show log level
        show_path=True, # Hide file path
        markup=True, # Enable Rich markup
    )

    # Set the logging level for the handler
    self._handler.setLevel(_log_level)

    # Configure the logging format globally
    custom_formatter = RichFormatter()
    self._handler.setFormatter(custom_formatter)
    # Add the handler to the logger
    if not self._logger.handlers:
        self._logger.addHandler(self._handler)
    self._logger.setLevel(_log_level)
    self._logger.propagate = False # Prevent propagation to root logger
    self._logger.debug(
        "Display initialized with logger '%s'." % self.logger_name)

def _validate_return(self, ret: Optional[Union[int, str]]) -> None:
    """Checks the return value of a database operation."""
    if isinstance(ret, int):
        if ret == 1:
            self._logger.error(
                "Database not initialized. Call init_db() first.")
            return
        if ret == 0:
            return
        self._logger.error("Unknown error occurred.")
        return
    if isinstance(ret, str):
        self._logger.error("Error: %s", ret)

def log_to_db(self, level: str, message: str) -> None:
    """
    Logs a message to the database.

```

```

:param level: The log level (e.g., INFO, DEBUG, ERROR).
:type level: str
:param message: The log message to save.
:type message: str
:return: None
"""
ret = LogDB.log_to_db(level, message)
self._validate_return(ret)

def info(self, message: str) -> None:
    """Logs an info message and saves it to the database."""
    self._logger.info(message)
    if self._log_to_db:
        self.log_to_db("INFO", message)

def debug(self, message: str) -> None:
    """Logs a debug message and saves it to the database."""
    self._logger.debug(message)
    if self._log_to_db:
        self.log_to_db("DEBUG", message)

def error(self, message: str) -> None:
    """Logs an error message and saves it to the database."""
    self._logger.error(message)
    if self._log_to_db:
        self.log_to_db("ERROR", message)

def warning(self, message: str) -> None:
    """Logs a warning message and saves it to the database."""
    self._logger.warning(message)
    if self._log_to_db:
        self.log_to_db("WARNING", message)

def critical(self, message: str) -> None:
    """Logs a critical message and saves it to the database."""
    self._logger.critical(message)
    if self._log_to_db:
        self.log_to_db("CRITICAL", message)

```

File: richdisplay/core/log_control.py

```

"""
LogControl class for managing logging operations.
"""

import argparse
import logging
import sys
from typing import Optional, Union
from rich.console import Console

from richdisplay.database import LogDB
from richdisplay.database.log_query import LogQuery
from richdisplay.exporters import CSVExporter, JSONExporter
from richdisplay.state import AuthManager, ArgsContext, SysContext

class LogControl:
    """Handles the logging control and management."""

    _instance: Optional["LogControl"] = None # Singleton instance
    _commands = [
        ["--clear", "-c"], "Clear the database.", bool, False, True],
        ["--db_path", "-dbp"], "Set the database path.", str, "logs.db", False],
        ["--db", "-db"], "Enable database logging.", bool, False, False],
        ["--debug", "-d"], "Enable debug mode.", bool, False, False],
        ["--get-logs", "-gl"], "Get logs from the database.", int, 0, True],
        ["--log-level", "-ll"], "Set the log level.", int, 10, True],
        ["--export", "-e"], "Export logs to a file.", str, "", True],
    ]

    _console: Console
    _bail_out = False
    _initialized: bool = False # Flag to check if the class is initialized

```

```

_logdb: LogDB = None # Instance of the LogDB class
_token: str = ""
_authorized_consumers = [] # List of authorized consumers

def __new__(cls, *args, **kwargs):
    """Ensures that only one instance of the class exists."""
    if cls._instance is None:
        cls._instance = super().__new__(cls)
        cls._instance._console = Console()
        cls._instance._console.print(
            "LogControl created.", style="bold green")
        cls._instance._initialize() # Perform initialization during creation
    if args:
        cls._instance._authorize(consumer=args[0]) # Authorize the consumer if provided
    return cls._instance

def _initialize(self):
    """Initializes the LogControl instance. Called only once."""
    # generate and store a token for this instance
    self._token = AuthManager.generate_token()

    if self._logdb is None:
        self._logdb = LogDB.init_db()
    if not self._initialized:
        self._console.print(
            "Initializing LogControl...", style="bold green")
        # parse CLI arguments and push to ArgsContext
        self.parse()

        self._initialized = True # Prevent reinitialization
        self._console.print(
            "LogControl initialized successfully.", style="bold green")

        # get the log level from ArgsContext
        log_level = ArgsContext.get().log_level

        # push new system context instead of contextmanager
        SysContext.push(
            auth_token=self._token,
            debug=True if log_level == logging.DEBUG else False,
            log_to_db=True,
            log_level=log_level,
        )

        # execute CLI commands under args context
        self.execute_commands()
def __init__(self, consumer: Optional[str] = None):
    """Prevent duplicate initialization."""
    if consumer:
        self._authorize(consumer=consumer)

def _exit(self, return_code=0):
    """Exits the program."""
    sys.exit(return_code)

@property
def bail_out(self) -> bool:
    """Checks if the program should exit."""
    return self._bail_out

@bail_out.setter
def bail_out(self, value: bool):
    """Sets the bail_out flag."""
    self._bail_out = value

def _authorize(self, consumer: str) -> Optional[Union[bool, str]]:
    """Authorizes the consumer to access the token."""
    if consumer not in self._authorized_consumers:
        self._authorized_consumers.append(consumer)
    return True
    return False

def _is_authorized(self, consumer: str) -> bool:
    """Checks if the consumer is authorized to access the token."""
    return consumer in self._authorized_consumers

def request_token(self, consumer: str) -> str:
    if not self._is_authorized(consumer):

```

```

        raise PermissionError(f"{consumer} not allowed to access token.")
    return self._token

@staticmethod
def is_initialized() -> bool:
    """Checks if the LogControl instance is initialized."""
    return LogControl._instance is not None

@staticmethod
def get_instance() -> "LogControl":
    """Returns the singleton instance of LogControl."""
    if LogControl._instance is None:
        LogControl._instance = LogControl()
    return LogControl._instance

def update_log_level(self, log_level: int) -> None:
    """Updates the log level in the ArgsContext."""
    ArgsContext.push(
        auth_token=self._token,
        log_level=log_level
    )
    SysContext.push(
        auth_token=self._token,
        debug=True if log_level == logging.DEBUG else False,
        log_to_db=True,
        log_level=log_level,
    )

def parse(self) -> None:
    """Parses CLI arguments and pushes them into ArgsContext."""
    parser = argparse.ArgumentParser(description="LogControl CLI")
    for cmd in self._commands:
        parser.add_argument(*cmd[0], help=cmd[1],
                             type=cmd[2], default=cmd[3])
    args = parser.parse_args()

    # Push parsed CLI flags into ArgsContext
    ArgsContext.push(
        auth_token=self._token,
        clear=args.clear,
        debug=args.debug,
        db=args.db,
        db_path=args.db_path,
        log_to_db=args.db,
        get_logs=args.get_logs,
        export=args.export,
        log_level=args.log_level
    )

def execute_commands(self):
    """Executes commands based on parsed arguments using ArgsContext."""
    # retrieve parsed CLI values from ArgsContext
    args = ArgsContext.get()

    if args.debug:
        self._console.print("Debug mode enabled.", style="bold yellow")
        self._console.print("Executing commands...", style="bold yellow")

    if args.clear:
        self._console.print("Clearing the database...", style="bold red")
        LogDB.clear_db()
        self._console.print("Database cleared.", style="bold green")
        self._exit()

    if args.log_to_db:
        self._console.print(
            f"Logging to database at {args.db_path}.", style="bold blue")

    if args.get_logs > 0:
        self._console.print(
            f"Fetching logs with level {args.get_logs}.", style="bold cyan")
        log_query = LogQuery(self._logdb)
        logs = log_query.get_logs_by_level(args.get_logs)
        ret = log_query.display_logs(logs, args.get_logs)
        if ret == 1:
            self._console.print(
                "Error fetching logs. Please check the log level.", style="bold red")
            self._exit(1)

```

```

        self._exit(0)

    if args.export:
        self._console.print(
            f"Exporting logs to {args.export}.", style="bold green")
        file_name = f"logs.{args.export}"
        self.export_logs(args.export, file_name)
        self._exit()

    # ArgsContext.pop() moved to Display.__del__ for proper lifecycle management

def export_logs(self, export_to="json", file_name="logs.json"):
    """Exports logs to a specified format (JSON or CSV)."""

    if export_to == "json":
        exporter = JSONExporter(self._logdb)
    elif export_to == "csv":
        exporter = CSVExporter(self._logdb)
    else:
        self._console.print(
            f"Unsupported export format: {export_to}.", style="bold red")
        return

    exporter.set_file_name(file_name)
    ret = exporter.export()
    if ret == 1:
        self._console.print(
            "Error exporting logs. Please check the file name.", style="bold red")
        return

    self._console.print(
        f"Logs exported successfully to {file_name}.", style="bold green")

```

File: richdisplay/database/__init__.py

```

from .log_db import LogDB

__all__ = [
    "LogDB",
]

```

File: richdisplay/database/log_db.py

```

"""
Handles logging messages to SQLite database.
"""

import os
import sqlite3
import psycopg2
from dataclasses import dataclass
from datetime import datetime
from typing import Optional, Union, Any

@dataclass
class BaseConnector:
    """Represents a base connector; not DB specific."""

    _connection = None

    def __init__(self, connection: Any):
        if connection:
            self._connection = connection
        else:
            raise ValueError("Connection cannot be None")

    def get_cursor(self):
        """Returns the cursor for the connection."""
        raise NotImplementedError("Subclasses must implement this method")

```

```

def close_connection(self):
    """Closes the connection."""
    raise NotImplementedError("Subclasses must implement this method")

def commit(self):
    """Commits the changes to the connection."""
    raise NotImplementedError("Subclasses must implement this method")

def rollback(self):
    """Rolls back the changes to the connection."""
    raise NotImplementedError("Subclasses must implement this method")

```

@dataclass

```

class SQLiteConnector(BaseConnector):
    """Represents a SQLite connector."""

    _db_path: str = None

    def __init__(self, connection: Any = None, db_path: str = None):
        if not connection:
            _connection = sqlite3.connect(db_path)
        else:
            _connection = connection
        super().__init__(connection=_connection)
        self._db_path = db_path

    def close_connection(self):
        """Closes the SQLite connection."""
        if self._connection:
            self._connection.close()
            self._connection = None
        else:
            raise ValueError("Connection is already closed or None")
        self._db_path = None

    def get_cursor(self):
        """Returns the cursor for the SQLite connection."""
        if self._connection:
            return self._connection.cursor()
        raise ValueError("Connection is already closed or None")

    def commit(self):
        """Commits the changes to the SQLite connection."""
        if self._connection:
            self._connection.commit()
        else:
            raise ValueError("Connection is already closed or None")

    def rollback(self):
        """Rolls back the changes to the SQLite connection."""
        if self._connection:
            self._connection.rollback()
        else:
            raise ValueError("Connection is already closed or None")

```

@dataclass

```

class PostgresConnector(BaseConnector):
    """Represents a PostgreSQL connector."""
    # Placeholder for PostgreSQL connection logic

    _db_host: str = None
    _db_port: int = None
    _db_name: str = None
    _db_user: str = None
    _db_password: str = None

    def __init__(self, connection: Any = None,
                  db_host: str = None, db_port: int = None,
                  db_name: str = None, db_user: str = None,
                  db_password: str = None):
        self._db_host = db_host
        self._db_port = db_port
        self._db_name = db_name
        self._db_user = db_user
        self._db_password = db_password
        if connection:

```

```

        super().__init__(connection=connection)
    else:
        # Establish a new PostgreSQL connection here
        try:
            _connection = psycopg2.connect(
                host=self._db_host,
                port=self._db_port,
                dbname=self._db_name,
                user=self._db_user,
                password=self._db_password
            )
            super().__init__(connection=_connection)
        except psycopg2.Error as e:
            raise ValueError(f"Failed to connect to PostgreSQL: {e}") from e

def get_cursor(self):
    """Returns the cursor for the PostgreSQL connection."""
    if self._connection:
        return self._connection.cursor()
    raise ValueError("Connection is already closed or None")

def close_connection(self):
    """Closes the PostgreSQL connection."""
    if self._connection:
        self._connection.close()
        self._connection = None
    else:
        raise ValueError("Connection is already closed or None")

def commit(self):
    """Commits the changes to the PostgreSQL connection."""
    if self._connection:
        self._connection.commit()
    else:
        raise ValueError("Connection is already closed or None")

def rollback(self):
    """Rolls back the changes to the PostgreSQL connection."""
    if self._connection:
        self._connection.rollback()
    else:
        raise ValueError("Connection is already closed or None")

```

```

@dataclass
class LogQuery:
    """Represents a log query"""
    _query: str
    _params: Optional[dict] = None

    def __init__(self, query: str, params: Optional[dict] = None):
        self._query = query
        self._params = params if params else {}
        self._params = {k: v for k, v in self._params.items() if v is not None}

    def __str__(self) -> str:
        """Returns the string representation of the query."""
        return self._query

    def __repr__(self) -> str:
        """Returns the string representation of the query."""
        return self.__str__()

    def __hash__(self) -> int:
        """Returns the hash of the query."""
        return hash((self._query, tuple(sorted(self._params.items()))))

    def execute_no_return(self, connection: SQLiteConnector) -> Optional[Union[int, str]]:
        """Executes the query on the given connection."""
        try:
            cursor = connection.get_cursor()
            cursor.execute(self._query, tuple(self._params.values()))
            connection.commit()
            return 0
        except sqlite3.Error as e:
            return e

    def execute(self, connection: SQLiteConnector) -> Optional[Union[int, list[tuple]]]:

```

```

    """Executes the query on the given connection and returns the result."""
    try:
        cursor = connection.get_cursor()
        cursor.execute(self._query, tuple(self._params.values()))
        return cursor.fetchall()
    except sqlite3.Error as e:
        return e

```

```

class LogDB:

```

```

    """Handles logging messages to SQLite."""

```

```

    _db_path = "logs.db"

```

```

    _connection = None

```

```

    @classmethod

```

```

    def clear_db(cls):

```

```

        """Clears the SQLite database."""

```

```

        if cls._connection:

```

```

            cls._connection.close()

```

```

            cls._connection = None

```

```

        try:

```

```

            os.remove(cls._db_path)

```

```

        except FileNotFoundError:

```

```

            pass

```

```

    @classmethod

```

```

    def init_db(cls) -> Optional[Union[int, str]]:

```

```

        """Initializes the SQLite database and creates the table if it doesn't exist."""

```

```

        try:

```

```

            cls._connection = SQLiteConnector(db_path=cls._db_path)

```

```

            query_string = """

```

```

                CREATE TABLE IF NOT EXISTS log_messages (

```

```

                    id INTEGER PRIMARY KEY AUTOINCREMENT,

```

```

                    level TEXT NOT NULL,

```

```

                    message TEXT NOT NULL,

```

```

                    timestamp TEXT NOT NULL

```

```

                )

```

```

            """

```

```

            params = {}

```

```

            _query = LogQuery(query_string, params)

```

```

            ack = _query.execute_no_return(cls._connection)

```

```

            if ack != 0:

```

```

                return ack

```

```

            return 0

```

```

        except sqlite3.Error as e:

```

```

            return e

```

```

    @classmethod

```

```

    def log_to_db(cls, level: str, message: str) -> Optional[Union[int, str]]:

```

```

        """Logs a message to the SQLite database."""

```

```

        if cls._connection is None:

```

```

            return 1

```

```

        timestamp = datetime.utcnow().isoformat()

```

```

        try:

```

```

            query_string = """

```

```

                INSERT INTO log_messages (level, message, timestamp)

```

```

                VALUES (?, ?, ?)

```

```

            """

```

```

            params = {

```

```

                "level": level,

```

```

                "message": message,

```

```

                "timestamp": timestamp

```

```

            }

```

```

            _query = LogQuery(query_string, params)

```

```

            ack = _query.execute_no_return(cls._connection)

```

```

            if ack != 0:

```

```

                return ack

```

```

            return 0

```

```

        except sqlite3.Error as e:

```

```

            return e

```

```

    @classmethod

```

```

    def fetch_all_logs(cls) -> Optional[Union[int, list[dict]]]:

```

```

        """Fetches all logs from the SQLite database."""

```

```

    if cls._connection is None:
        return 1

    try:
        query_string = "SELECT * FROM log_messages"
        params = {}
        _query = LogQuery(query_string, params)
        rows = _query.execute(cls._connection)
        if isinstance(rows, int):
            return rows
        logs = [
            {"id": row[0], "level": row[1],
             "message": row[2], "timestamp": row[3]}
            for row in rows
        ]
        return logs
    except sqlite3.Error as e:
        return e

@classmethod
def fetch_logs_by_level(cls, level: str) -> Optional[Union[int, list[dict]]]:
    """Fetches logs by level from the SQLite database."""
    if cls._connection is None:
        return 1

    try:
        query_string = "SELECT * FROM log_messages WHERE level = ?"
        params = {"level": level}
        _query = LogQuery(query_string, params)
        rows = _query.execute(cls._connection)
        if isinstance(rows, int):
            return rows
        logs = [
            {"id": row[0], "level": row[1],
             "message": row[2], "timestamp": row[3]}
            for row in rows
        ]
        return logs
    except sqlite3.Error as e:
        return e

@classmethod
def close_db(cls) -> None:
    """Closes the SQLite database connection."""
    if cls._connection:
        cls._connection.close()
        cls._connection = None

```

File: richdisplay/database/log_query.py

```

"""
Log Query Module
"""

from typing import Optional, Union, List

from rich.console import Console
from rich.table import Table

from richdisplay.database import LogDB

class LogQuery:
    """
    Class for querying logs from the database.
    """

    _lvl_mapping = {
        10: {
            "name": "DEBUG",
            "description": "Debug level logs.",
            "func": "get_debug_logs",
        },
        20: {

```

```

        "name": "INFO",
        "description": "Info level logs.",
        "func": "get_info_logs",
    },
    30: {
        "name": "ERROR",
        "description": "Error level logs.",
        "func": "get_error_logs",
    },
    40: {
        "name": "WARNING",
        "description": "Warning level logs.",
        "func": "get_warning_logs",
    },
    50: {
        "name": "CRITICAL",
        "description": "Critical level logs.",
        "func": "get_critical_logs",
    }
}

def __init__(self, db: LogDB):
    """Initialize with a database instance."""
    self._db = db

def get_logs(self, limit: int = 10) -> List[dict]:
    """Retrieve logs from the database with an optional limit."""

    logs = self._db.fetch_all_logs()
    if isinstance(logs, list):
        return logs[:limit]
    return []

def get_logs_by_level(self, level: int) -> Optional[Union[int, list[dict]]]:
    """
    Retrieve logs by level from the database.

    :param level: The log level to filter by.
    :return: A list of logs matching the specified level.
    """
    if level not in self._lvl_mapping:
        return 1

    level: str = ""
    if level in self._lvl_mapping:
        level = self._lvl_mapping[level]["name"]
    else:
        return 1

    return self._db.fetch_logs_by_level(level)

def display_logs(self, logs: Optional[Union[int, list[dict]]], level: str) -> Optional[Union[int, str]]:
    """Displays logs in a formatted table."""
    if isinstance(logs, int):
        return 1

    _console = Console()
    if logs:
        table = Table(title=f"Log Messages - {level.capitalize()}")
        table.add_column("ID", justify="right")
        table.add_column("Level", style="cyan")
        table.add_column("Message", style="magenta")
        table.add_column("Timestamp", style="green")

        for log in logs:
            table.add_row(
                str(log["id"]),
                log["level"],
                log["message"],
                log["timestamp"]
            )

        _console.print(table)
        return 0
    _console.print(
        f"No logs found for level: {level}", style="bold yellow"
    )
    return "no_logs"

```

File: richdisplay/utls/__init__.py

```
from .formatter import RichFormatter
```

```
__all__ = [  
    "RichFormatter",  
]
```

File: richdisplay/utls/formatter.py

```
"""  
Custom Formatter for logging with rich library.  
"""  
  
import logging  
  
class RichFormatter(logging.Formatter):  
    """Custom Formatter to add color to the logger name."""  
  
    _format_string = "%(asctime)s - %(name)s - %(module)s:%(lineno)d - %(message)s"  
  
    def format(self, record):  
        # Rich-Markup für den logger name hinzufügen  
        record.name = f"[bold blue]{record.name}[/]"  
  
        formatter = logging.Formatter(self._format_string)  
        formatted_record = formatter.format(record)  
  
        return formatted_record
```

File: richdisplay/exporters/__init__.py

```
"""  
Init routine for the exporters module.  
"""  
  
from .csv_exporter import CSVExporter  
from .json_exporter import JSONExporter  
  
__all__ = [  
    "CSVExporter",  
    "JSONExporter",  
]
```

File: richdisplay/exporters/csv_exporter.py

```
"""  
Class to export data to CSV format.  
"""  
  
import csv  
from typing import List, Dict, Optional, Union, Any  
  
class CSVExporter:  
    """  
    Class to export data to CSV format.  
    """  
  
    _file_name: str = ""
```

```

_data: List[Dict[str, Any]] = []
_encoding: str = "utf-8"

def __init__(self, encoding: str = "utf-8") -> None:
    """
    Initialize the CSVExporter with the specified encoding.

    :param encoding: The encoding to use for the CSV file.
    """
    self._encoding = encoding

@classmethod
def set_file_name(cls, file_name: str) -> str:
    """
    Set the file name for the CSV export.

    :param file_name: The desired file name.
    :return: The formatted file name.
    """

    cls._file_name = file_name
    return f"{cls._file_name}.csv"

@classmethod
def get_file_name(cls) -> str:
    """
    Get the file name for the CSV export.

    :return: The file name.
    """
    return cls._file_name

@classmethod
def set_data(cls, data: List[Dict[str, Any]]) -> bool:
    """
    Set the data to be exported to CSV.

    :return: 0 if successful, 1 if no data is available.
    """
    if not data:
        return False
    cls._data = data
    return True

@classmethod
def get_data(cls) -> Optional[Union[int, List[Dict[str, Any]]]]:
    """
    Get the data to be exported to CSV.

    :return: The data to be exported.
    """
    if not cls._data:
        return 1
    return cls._data

@classmethod
def export(cls, file_name: str = None) -> Optional[Union[int, str]]:
    """
    Export the data to a CSV file.

    :param file_name: The name of the CSV file.
    :return: 0 if successful, 1 if no data is available.
    """
    if not cls._data:
        return 1
    if file_name is None:
        file_name = cls._file_name
    with open(file_name, mode="w", encoding=cls._encoding) as csv_file:
        writer = csv.DictWriter(csv_file, fieldnames=cls._data[0].keys())
        writer.writeheader()
        writer.writerows(cls._data)
    return 0

```

File: richdisplay/exporters/json_exporter.py

```
"""
JSONExporter class for exporting logs in JSON format.
"""
```

```
from typing import List, Dict, Any, Optional, Union
import json
```

```
class JSONExporter:
    """
    Class to export data to JSON format.
    """

    _file_name: str = ""
    _data: List[Dict[str, Any]] = []
    _encoding: str = "utf-8"

    def __init__(self, encoding: str = "utf-8") -> None:
        """
        Initialize the JSONExporter with the specified encoding.

        :param encoding: The encoding to use for the JSON file.
        """
        self._encoding = encoding

    @classmethod
    def set_file_name(cls, file_name: str) -> str:
        """
        Set the file name for the JSON export.

        :param file_name: The desired file name.
        :return: The formatted file name.
        """
        cls._file_name = file_name
        return f"{cls._file_name}.json"

    @classmethod
    def get_file_name(cls) -> str:
        """
        Get the file name for the JSON export.

        :return: The file name.
        """
        return cls._file_name

    @classmethod
    def set_data(cls, data: List[Dict[str, Any]]) -> bool:
        """
        Set the data to be exported to JSON.

        :return: 0 if successful, 1 if no data is available.
        """
        if not data:
            return False
        cls._data = data
        return True

    @classmethod
    def get_data(cls) -> Optional[Union[int, List[Dict[str, Any]]]]:
        """
        Get the data to be exported to JSON.

        :return: The data to be exported.
        """
        if not cls._data:
            return 1
        return cls._data

    @classmethod
    def export(cls) -> Optional[Union[int, str]]:
        """
        Export the data to a JSON file.

        :return: 0 if successful, 1 if no data is available.
        """
        if not cls._data:
            return 1
```

```
try:
    with open(cls._file_name, "w", encoding=cls._encoding) as json_file:
        json.dump(cls._data, json_file, indent=4)
    return 0
except Exception as e: # pylint: disable=broad-except
    return str(e)
```